

---

INFRASTRUCTURE SPECIFICATION v2 | MARCH 2026

# Babylon Deployment Infrastructure

Terraform, Ansible, and Bare Metal  
for a Solo-Developer Political Economy Simulation

---

Percy Raskova  
*Claude Opus*

---

*“A backup never restored is a hypothesis, not a backup.”*

— Operations proverb

*“You end up with a versioned, documented, repeatable description of your VPS state instead of a half-remembered sequence of shell commands.”*

— On Ansible, this project, March 2026

## 1. Overview

---

Babylon is an AGPL-3.0 Django/Postgres political economy simulation engine with in-memory NetworkX graph processing, totaling approximately 86,000 lines of Python. It is transitioning from a PyQt6 desktop application to a server-client architecture: Django + Postgres server behind a React web frontend. This specification defines the production infrastructure for beta deployment, optimized for a solo developer shipping a real product on a constrained budget.

The governing design principle is **everything runs bare metal, managed by Ansible**. There is no Docker, no Nix, no containerization layer. Terraform provisions infrastructure and seeds the initial deploy user via cloud-init. Ansible configures and maintains everything on the server. systemd supervises all processes.

### Design Constraint

This is a solo-developer deployment. Every architectural decision is filtered through: will this require a second full-time job to maintain? If yes, reject it. Complexity is the enemy.

### Changes from v1

Version 1 of this spec used Docker for Postgres and Nix for application environment management. Both have been removed. Postgres 18 installs from PGDG apt packages and is managed by Ansible's `community.postgresql` collection. Python and Poetry install via system packages. The entire server state is described by Ansible playbooks. cloud-init in Terraform handles the initial passwordless handshake for Ansible to take over.

## 2. Compute Provider: Hetzner Cloud

---

### 2.1 Selection Rationale

Five providers were evaluated: Hetzner, DigitalOcean, OVH, Contabo, and Alibaba Cloud.

| Provider      | 4C/8GB Equiv. | Terraform | Verdict             |
|---------------|---------------|-----------|---------------------|
| Hetzner CX32  | ~\$8.50/mo    | Official  | <b>Selected</b>     |
| DigitalOcean  | \$24/mo       | Official  | 4–5× premium        |
| Contabo VPS S | ~\$6.50/mo    | None      | Oversold, CPU steal |
| OVH           | ~\$12/mo      | Partial   | Weak support, UX    |
| Alibaba Cloud | Variable      | Partial   | Asia-focused        |

---

Hetzner was selected on the basis of price-performance leadership (rated #1 global VPS in 2025/2026 benchmarks), official Terraform provider support (`hetznercloud/hcloud`), hourly billing with instant provisioning, 20 TB included monthly traffic, and availability of a US East (Ashburn, VA) datacenter providing low latency to the Detroit test geography.

Contabo was rejected despite the lowest sticker price due to documented CPU oversubscription (high CPU steal under load), slow provisioning, poor support response times, and the absence of an official Terraform provider. DigitalOcean was rejected on cost: a 4–5× premium for equivalent specs.

## 2.2 Selected Instance

| Parameter     | Value                   |
|---------------|-------------------------|
| Instance type | CX32 (shared vCPU, x86) |
| vCPUs         | 4                       |
| RAM           | 8 GB                    |
| Storage       | 80 GB NVMe              |
| Traffic       | 20 TB/month included    |
| Location      | ash (Ashburn, Virginia) |
| Image         | debian-12               |
| Cost          | €8/month (\$8.50 USD)   |

## 2.3 Hetzner Services Used

**Cloud Firewall.** L3/L4 stateful packet filtering applied at the hypervisor level before traffic reaches the VPS. Inbound locked to Cloudflare IP ranges (HTTP/HTTPS) plus SSH from known IPs.

**Volumes.** Attached NVMe block storage for Postgres data if the base 80 GB is outgrown.

**Snapshots and Backups.** Automated server snapshots for disaster recovery, approximately 20% of server price. Supplements application-level Postgres backups.

**Private Networks.** RFC 1918 address space for future multi-server expansion.

## 2.4 Hetzner Services Not Used

**Object Storage.** EU-only locations (Falkenstein, Nuremberg, Helsinki). With VPS in Ashburn, transatlantic round trips create unacceptable latency. Cloudflare R2 replaces this.

**Load Balancers.** Single-server beta. Cloudflare provides L7 load balancing if needed later.

## 3. Edge Services: Cloudflare

---

---

Cloudflare operates as the shield and content delivery layer. Nothing computes on Cloudflare; nothing reaches users without passing through Cloudflare first. The domain `babylon.percypedia.biz` is DNS-proxied through Cloudflare.

### 3.1 Free Tier Services

**DNS + SSL Termination.** Authoritative DNS with automatic HTTPS certificate provisioning at the edge. The public internet never sees the Hetzner IP directly.

**DDoS Protection + WAF.** L7 HTTP inspection, SQL injection blocking, rate limiting. Hetzner’s Cloud Firewall operates at L3/L4 only—Cloudflare provides the application-layer security.

**CDN Caching.** React build artifacts and static assets served from edge.

**Uptime Monitoring.** Built-in health checks for proxied domains.

### 3.2 Paid Services

**R2 Object Storage.** Zero egress fees, S3-compatible. Used for Postgres backup archives and static media. Free tier provides 10 GB storage, 1M writes, and 10M reads per month.

**Workers AI.** Serverless inference for narrative generation via LoRA-adapted 8B models. Approximately \$0.00014 per narration call, \$0.028 per full game session. Handles the “AI Observes, Never Controls” narrative layer.

### 3.3 Division of Labor

#### Governing Principle

Cloudflare = shield + content delivery. Hetzner = brain. Nothing computes on Cloudflare; nothing reaches users without Cloudflare first.

---

| Function              | Owner       | Notes               |
|-----------------------|-------------|---------------------|
| DNS, SSL, DDoS, WAF   | Cloudflare  | Free tier           |
| CDN / static assets   | Cloudflare  | Free tier           |
| Object storage (R2)   | Cloudflare  | Zero egress         |
| Narrative AI          | Cloudflare  | Workers AI          |
| L3/L4 firewall        | Hetzner     | Cloud Firewall      |
| Nginx reverse proxy   | Hetzner VPS | Ansible-managed     |
| Django + Gunicorn     | Hetzner VPS | systemd service     |
| PostgreSQL 18         | Hetzner VPS | PGDG bare metal     |
| NetworkX graph engine | Hetzner VPS | In-memory, per-tick |
| CI/CD (Woodpecker)    | Hetzner VPS | systemd service     |

## 4. Operating System: Debian 12 (Bookworm)

---

Debian 12 (Bookworm) is the base operating system, supported through June 2028. Selected over Ubuntu for the absence of snap daemon overhead, no Canonical telemetry, and intentionally conservative package versions—a feature for servers.

Terraform provisions the server with `image = "debian-12"`. Ansible manages all packages, configuration, and services from first boot onward.

## 5. Bootstrapping: Terraform + cloud-init

---

The bootstrap problem: Terraform creates a server, but Ansible needs SSH access to configure it. Hetzner Cloud supports cloud-init<sup>1</sup>, which solves the initial handshake.

### 5.1 What cloud-init Does

Terraform passes a cloud-init configuration to `hc_loud_server` via the `user_data` argument. On first boot, cloud-init:

1. Creates a non-root deploy user (e.g., `deploy`)
2. Installs the SSH public key for passwordless access

---

<sup>1</sup> cloud-init is an industry-standard tool for early server initialization. It runs once on first boot, before any human or automation tool connects.

3. Grants passwordless sudo to the deploy user
4. Disables root SSH login and password authentication
5. Optionally installs Python (required for Ansible to run)

This is the *only* thing cloud-init does. It creates the door that Ansible walks through. All subsequent configuration is Ansible's responsibility.

## 5.2 Terraform Resource

```
resource "hcloud_server" "babylon" {
  name          = "babylon-prod"
  server_type   = "cx32"
  location      = "ash"
  image         = "debian-12"
  ssh_keys      = [hcloud_ssh_key.deploy.id]

  user_data = templatefile("cloud-init.yml", {
    deploy_user      = var.deploy_user
    deploy_ssh_key    = var.deploy_ssh_pubkey
  })
}
```

After `terraform apply` completes and the server is online, Ansible connects as the deploy user and takes over entirely.

## 6. Configuration Management: Ansible

---

Ansible is the single source of truth for the entire server state. Every package, configuration file, systemd unit, firewall rule, and cron job is described in Ansible playbooks and roles. Nothing is installed or configured by hand on the VPS.

### 6.1 Ansible Roles

#### 6.1.1 common

OS-level hardening and baseline configuration:

1. Harden SSH: key-only auth, disable password login, disable root login (reinforces cloud-init)
  2. Install and configure `fail2ban`
  3. Enable `unattended-upgrades` for Debian security patches
-

4. Configure `nftables` host firewall (defense-in-depth behind Hetzner Cloud Firewall)
5. Create 2 GB swap file (prevents OOM killer from assassinating Postgres under memory spikes)

### 6.1.2 postgres

Installs and configures PostgreSQL 18 from PGDG bare metal. Uses the `community.postgresql` Ansible collection<sup>2</sup>:

1. Add PGDG apt repository and signing key
2. Install `postgresql-18`, `postgresql-18-postgis-3`, `postgresql-18-pgvector`
3. Create the `babylon` database role with `scram-sha-256` password authentication
4. Create the `babylon` database owned by that role
5. Enable extensions: `postgis`, `vector`, `gen_random_uuid`
6. Configure `pg_hba.conf`: local password auth for the `babylon` role, localhost only
7. Tune `postgresql.conf` for 8 GB RAM (`shared_buffers`, `work_mem`, `effective_cache_size`)
8. Reload Postgres (handler triggered only when config changes)

Postgres listens only on localhost. It is never exposed to the internet. Django connects via Unix socket for zero TCP overhead.

### 6.1.3 python

1. Install Python 3.12+ from Debian repos (or deadsnakes PPA if Bookworm ships older)
2. Install `pipx` and use it to install Poetry globally
3. Clone the Babylon repository to `/opt/babylon`
4. Run `poetry install --no-dev` in the project directory

### 6.1.4 babylon

1. Deploy `babylon.service` systemd unit (Jinja2 template, see Section 7)
2. Deploy Django `.env` with production settings: `DEBUG=False`, database credentials, secret key, allowed hosts
3. Run `python manage.py collectstatic`
4. Run `python manage.py migrate`

---

<sup>2</sup> Install with `ansible-galaxy collection install community.postgresql`. Provides idempotent modules for database creation, role management, extension enabling, and `pg_hba` configuration.

5. Enable and start the service

#### 6.1.5 nginx

1. Install Nginx from Debian repos
2. Deploy site configuration: reverse proxy to Gunicorn on `127.0.0.1:8000`
3. Serve static files from Django's `STATIC_ROOT` directly
4. Configure upstream connection buffering
5. With Cloudflare proxied: accept traffic from Cloudflare IP ranges, set `real_ip_header` to `CF-Connecting-IP`

#### 6.1.6 woodpecker

1. Download Woodpecker server and agent binaries
2. Deploy systemd units for both
3. Configure Codeberg OAuth integration

#### 6.1.7 backup

1. Deploy `backup-postgres.sh` script
2. Install `rc1one` for R2 uploads
3. Create cron job: daily `pg_dump`, compress, upload to R2
4. Retention: 7 daily + 4 weekly via R2 lifecycle rules

### 6.2 Idempotency and Drift Correction

Every Ansible task is idempotent. Running the playbook against an already-configured server changes nothing. Weekly playbook runs (cron or manual) detect and correct configuration drift. The `notify + handler` pattern ensures services restart only when their configuration actually changes.

## 7. Process Supervision: systemd

---

systemd is PID 1 on Debian and handles all process lifecycle management. No additional supervisor is needed.

---

## 7.1 Supervision Tree

```
systemd (PID 1)
|-- postgresql@18-main.service      (PGDG-managed)
|-- babylon.service
|   +-- gunicorn (3 workers) -> Django + NetworkX
|-- nginx.service
|-- woodpecker-server.service
|-- woodpecker-agent.service
|-- sshd.service
|-- fail2ban.service
+-- unattended-upgrades.timer
```

## 7.2 Babylon Service Unit

The `babylon.service` unit file is deployed by Ansible as a Jinja2 template with variables derived from the target host. Key properties:

**Restart policy.** `Restart=always` with `RestartSec=5` keeps Gunicorn alive through crashes.

**Dependency ordering.** `After=postgresql@18-main.service` ensures Postgres is ready before Babylon starts.

**Resource controls.** cgroup limits: `MemoryMax=6G` and `CPUWeight=80` prevent Babylon from starving Postgres during heavy graph operations.

**Logging.** `stdout/stderr` captured by the journal. Query with `journalctl -u babylon`. No log rotation configuration required.

## 7.3 Request Path

The full production request path:

```
Player -> Cloudflare (SSL, WAF, CDN)
        -> Hetzner Firewall (L3/L4)
        -> Nginx (reverse proxy, static files)
        -> Gunicorn (3 workers)
        -> Django
        -> Postgres (Unix socket)
```

For 15 beta testers playing turn-based, 2–3 Gunicorn workers is sufficient. Configuration is one line: `gunicorn babylon.wsgi:application --workers 3 --bind 127.0.0.1:8000`.

## 8. Database: PostgreSQL 18 (Bare Metal)

---

---

PostgreSQL 18 installs from the PGDG apt repository directly on the host OS. No Docker container. No version indirection. Ansible’s `community.postgresql` collection handles the configuration declaratively.

8.1 Extensions

| Extension | Purpose                                          |
|-----------|--------------------------------------------------|
| PostGIS   | Spatial queries, county boundaries, H3 hex grids |
| pgvector  | Embedding/vector storage for narrative AI RAG    |
| pgrouting | In-database graph routing on spatial data        |
| pgtap     | Database-level unit testing                      |

Apache AGE (Cypher graph queries) is deferred. Recursive CTEs handle graph traversal for beta. SQL/PGQ (ISO SQL:2023 graph queries) is the long-term direction as Postgres implements the standard natively.

8.2 Connection Architecture

Django connects to Postgres via Unix socket (`/var/run/postgresql/`), not TCP. This eliminates TCP overhead and simplifies security—no port to firewall, no network exposure. The `pg_hba.conf` entry is `local babylon babylon scram-sha-256`.

8.3 Data Architecture

The SQLite reference database (ingested federal data: QCEW, BEA, Census, FRED, HIFLD, BTS, FCC, ATUS) remains the immutable ground truth for calibration and validation. It is uploaded to the VPS and consulted during simulation initialization. Postgres handles all runtime state: game sessions, simulation ticks, graph persistence, user accounts, and training data collection.

Completed games export to Parquet (zstd compression, ~10:1 ratio), upload to R2, and purge from Postgres. DuckDB reads Parquet from R2 natively for cross-game analytics.

9. Infrastructure Provisioning: Terraform

---

Terraform manages cloud infrastructure as declarative code. Two providers:

**hetznercloud/hcloud** (official partner). Servers, cloud firewalls, SSH keys, networks, volumes.

**cloudflare/cloudflare** (official). DNS records, R2 buckets, page rules, WAF configuration.

---

## 9.1 Terraform Resources

| Resource                     | Provider   |
|------------------------------|------------|
| hcloud_server (+ cloud-init) | Hetzner    |
| hcloud_firewall              | Hetzner    |
| hcloud_ssh_key               | Hetzner    |
| hcloud_network               | Hetzner    |
| cloudflare_record            | Cloudflare |
| cloudflare_r2_bucket         | Cloudflare |
| cloudflare_ruleset           | Cloudflare |

Terraform's *only job* is creating and destroying cloud resources. It does not configure what runs on them—that is Ansible's domain. The `user_data` (cloud-init) argument on `hcloud_server` is the single bridge: it creates the `deploy` user so Ansible can connect.

`terraform.tfvars` is gitignored. State stored locally for solo operation.

## 10. Secrets Management

### 10.1 Current Approach

**ansible-vault** encrypts variable files containing sensitive values. Decryption key stored on the operator's machine, never committed.

**.env files** (mode 0600) deployed by Ansible to the VPS for Django settings. A `.env.template` is committed with placeholders; the actual `.env` is gitignored.

### 10.2 Secrets Inventory

| Secret                       | Consumers                     |
|------------------------------|-------------------------------|
| Cloudflare API token         | Terraform, backup script      |
| R2 credentials               | Backup script, Django storage |
| Workers AI token             | Django narrative layer        |
| Postgres password            | Ansible, Django settings      |
| Django SECRET_KEY            | Django settings               |
| Woodpecker OAuth credentials | Woodpecker server             |

### 10.3 Upgrade Path

**SOPS + age** when the secrets inventory grows. SOPS encrypts individual values within YAML (meaningful diffs in version control). Age provides simple public-key encryption without GPG. HashiCorp Vault is explicitly rejected as over-engineered for solo operation.

## 11. Backup Strategy

---

### 11.1 Postgres Backups

1. `pg_dump` via cron produces a SQL dump
2. Compress with `zstd`
3. Upload to Cloudflare R2 via `rc1one`
4. Retain 7 daily + 4 weekly backups via lifecycle policy

### 11.2 Restore Verification

*A backup never restored is a hypothesis, not a backup.*

Monthly restore verification runs as a Woodpecker pipeline: pull the latest backup from R2, create a temporary Postgres database on the same server, restore the dump, run row-count and schema-integrity checks, drop the temporary database, report pass/fail. Failure is treated as a production incident.

### 11.3 Server-Level Backups

Hetzner automated snapshots (~20% of server cost) provide full-disk recovery. These supplement but do not replace `pg_dump`—snapshots capture a potentially inconsistent database state.

## 12. Monitoring and Logging

---

### 12.1 Beta-Appropriate Monitoring

**Cloudflare uptime checks.** Built-in for proxied domains. Zero configuration.

**Disk usage alert.** Cron job checking `df -h`, alerting when any filesystem exceeds 80%.

---

## 12.2 What Is Explicitly Rejected

Prometheus + Grafana + Loki is a second full-time job. For one server, `journalctl` combined with Cloudflare analytics provides sufficient observability.

## 12.3 Logging Architecture

Django logs JSON to stdout. Gunicorn and Nginx log to the journal or to standard log files under `/var/log/`.

The v1 debugging workflow: `journalctl -u babylon` for application errors, Nginx error logs for proxy issues. Centralized log aggregation is a future problem.

# 13. CI/CD: Woodpecker

---

Woodpecker server and agent run as systemd services (standalone binaries, no Docker). Woodpecker monitors the Codeberg repository and triggers pipelines on push to `main`.

## 13.1 Deploy Pipeline

1. Woodpecker detects push to `main`
2. Run test suite
3. `git pull origin main` (Woodpecker runs on the same host)
4. `poetry install --no-dev`
5. `python manage.py migrate`
6. `python manage.py collectstatic --noinput`
7. `sudo systemctl restart babylon`

## 13.2 Additional Pipelines

**Backup verification.** Monthly scheduled pipeline (Section 11).

**Lint and type-check.** `ruff`, `mypy`, formatting checks.

# 14. Source Code Hosting: Codeberg

---

---

Codeberg, a nonprofit Git forge. AGPL-3.0 licensing with commercial SaaS hosting (the Red Hat model: free source, paid convenience tier at \$20/month) is fully compliant with Codeberg's terms. Private repos permitted for FOSS contributors (security patches, release prep).

## 15. Repository Structure

---

```
babylon/
|-- infra/
|   |-- terraform/
|   |   |-- main.tf           # Hetzner server + cloud-init
|   |   |-- cloudflare.tf     # DNS, R2 buckets, page rules
|   |   |-- variables.tf
|   |   |-- outputs.tf        # Server IP, bucket URLs
|   |   |-- cloud-init.yml     # Deploy user bootstrap
|   |   +-- terraform.tfvars  # (gitignored)
|   |-- ansible/
|   |   |-- ansible.cfg
|   |   |-- inventory.yml     # Populated from terraform output
|   |   |-- playbook.yml      # Master playbook
|   |   |-- roles/
|   |   |   |-- common/       # SSH, fail2ban, firewall, swap
|   |   |   |-- postgres/     # PGDG install, extensions, config
|   |   |   |-- python/       # Python 3.12, Poetry
|   |   |   |-- babylon/      # App deploy, systemd unit, .env
|   |   |   |-- nginx/        # Reverse proxy config
|   |   |   |-- woodpecker/    # CI/CD binaries + systemd
|   |   |   +-- backup/       # pg_dump cron, rclone to R2
|   |   +-- vault/secrets.yml  # ansible-vault encrypted
|   +-- scripts/
|       +-- upload-reference-sqlite.sh
+-- src/babylon/
```

## 16. Deployment Workflow

---

## 16.1 Initial Provisioning (Day 1)

1. `terraform apply` creates the Hetzner server (with cloud-init deploy user), cloud firewall, SSH key, private network, and Cloudflare DNS records and R2 buckets
2. Terraform outputs the server IP address
3. Wait for cloud-init to complete (~30 seconds)
4. `ansible-playbook playbook.yml` connects as the deploy user and configures everything: OS hardening, Postgres 18 with extensions, Python/Poetry, Nginx, Gunicorn, Woodpecker, backup cron, systemd units
5. Upload the reference SQLite database
6. Verify: `curl https://babylon.percypedia.biz/health/` returns 200

## 16.2 Subsequent Deploys

Woodpecker handles routine deploys on push to `main`: pull, install, migrate, collectstatic, restart.

## 16.3 Drift Correction

Running `ansible-playbook playbook.yml` periodically ensures configuration has not drifted. Ansible is idempotent—unchanged resources are skipped.

# 17. Cost Analysis

---

| Service                | Monthly Cost       | Notes              |
|------------------------|--------------------|--------------------|
| Hetzner CX32           | ~\$8.50            | Compute            |
| Hetzner backups        | ~\$1.70            | 20% of server      |
| Cloudflare (free tier) | \$0                | DNS, CDN, WAF      |
| Cloudflare R2          | \$0                | Free tier for beta |
| Cloudflare Workers AI  | ~\$3–5             | Usage-dependent    |
| Codeberg               | \$0                | Free for FOSS      |
| <b>Total (beta)</b>    | <b>~\$13–15/mo</b> |                    |

# 18. Security Posture

---

---

## 18.1 Network Security (Defense in Depth)

1. **Cloudflare WAF** — L7 inspection, DDoS, rate limiting
2. **Hetzner Cloud Firewall** — L3/L4 stateful rules at the hypervisor, before traffic reaches the VPS
3. **Host firewall (nftables)** — Defense-in-depth on the VPS itself
4. **fail2ban** — Bans IPs after repeated SSH failures
5. **Nginx** — `real_ip_header CF-Connecting-IP`, only accepts connections from Cloudflare ranges

## 18.2 Access Control

SSH key-only authentication. Password login disabled. Root login disabled. Dedicated deploy user with scoped sudo. Django admin for player account creation—no self-registration for beta.

## 18.3 Application Security

`DEBUG=False` in production. SSL redirect, secure session cookies, secure CSRF cookies, and HSTS all enabled. Postgres listens only on Unix socket. `ALLOWED_HOSTS` restricted to the production domain.

## 18.4 Patching

unattended-upgrades handles Debian security patches automatically. Application dependencies updated deliberately via `poetry update` and tested in CI before deploy.

## 19. Rejected Alternatives

---

---

| Technology                  | Reason for Rejection                        |
|-----------------------------|---------------------------------------------|
| Docker                      | Unnecessary indirection for one server      |
| Nix / NixOS                 | Complexity tax without proportional benefit |
| Kubernetes                  | Cluster orchestration for one server        |
| Consul / service mesh       | Service discovery for one service           |
| HashiCorp Vault             | Key management for six secrets              |
| Packer                      | Image baking for one server type            |
| ArgoCD / Flux               | GitOps for one deployment target            |
| Prometheus + Grafana + Loki | Full observability for one VPS              |
| Datadog / New Relic         | SaaS monitoring cost for beta               |
| Ubuntu                      | Snap overhead, Canonical telemetry          |
| Contabo                     | CPU oversubscription, no Terraform          |

Each of these solves real problems at scale. None of them solve problems Babylon has today.



## 20. Future Considerations

---

**Multi-server split.** When Postgres and the simulation engine need dedicated resources, Terraform provisions a second Hetzner server on the private network. Ansible roles already isolate concerns by service.

**Federated instances.** Each federation node is an independent Hetzner VPS running the same Terraform/Ansible configuration.

**Remote Terraform state.** If a second operator joins, migrate to a shared backend (Terraform Cloud free tier or R2).

**SOPS migration.** When secrets inventory grows beyond six values.

**Monitoring upgrade.** Victoria Metrics (single-binary Prometheus alternative) before considering Grafana.

**Apache AGE / SQL:PGQ.** Graph queries inside Postgres, replacing recursive CTEs when the extension or standard matures.

---

*“Every piece has exactly one job  
and nothing overlaps.”*

Babylon Infrastructure Design Principle